

✓ Logistic Regression and Sentiment analysis

In this assignment you will implement and experiment with various feature engineering techniques in the context of Logistic Regression models for Sentiment classification of movie reviews.

1. Read lexicons of positive and negative sentiment words.
2. Implement overall positive and negative lexicon count features.
3. Implement per-lexicon-word count features.
4. Implement document length feature.
5. Implement deictic features.
6. Analysis of results.
7. Bonus points.

We will use the LR model implemented in sklearn:

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

Write Your Name Here: Kaspian Thomas

✓ Submission Instructions

1. Click the Save button at the top of the Jupyter Notebook/Google Colab.
2. Please make sure to have entered your name above.
3. Select Cell -> All Output -> Clear. This will clear all the outputs from all cells (but will keep the content of all cells).
4. Select Cell -> Run All. This will run all the cells in order, and will take several minutes.
5. Once you've rerun everything, select File -> Download as/Print -> PDF and download a PDF version *LR-SentimentAnalysis.pdf* showing the code and the output of all cells, and save it in the same folder that contains the notebook file *LR-SentimentAnalysis.ipynb*.
6. Look at the PDF file and make sure all your solutions are there, displayed correctly. The PDF is the only thing we will see when grading!
7. Submit **both** your PDF and notebook on Canvas.
8. Verify your Canvas submission contains the correct files by downloading it after posting it on Canvas.

✓ From documents to feature vectors

This section illustrates the prototypical components of machine learning pipeline for an NLP task, in this case document classification:

1. Read document examples (train, devel, test) from files with a predefined format:
 - assume one document per line, using the format "<label> <text>".
2. Tokenize each document:
 - using a spaCy tokenizer.
3. Feature extractors:
 - so far, just words.
4. Process each document into a feature vector:
 - map document to a dictionary of feature names.
 - map feature names to unique feature IDs.
 - each document is a feature vector, where each feature ID is mapped to a feature value (e.g. word occurrences).

```
import spacy
from spacy.lang.en import English
from scipy import sparse
from sklearn.linear_model import LogisticRegression
```

```
# Create spaCy tokenizer.
spacy_nlp = English()
```

```
def spacy_tokenizer(text):
    tokens = spacy_nlp.tokenizer(text)

    return [token.text for token in tokens]
```

```
def read_examples(filename):
    X = []
    Y = []
    with open(filename, mode = 'r', encoding = 'utf-8') as file:
        for line in file:
            [label, text] = line.rstrip().split(' ', maxsplit = 1)
            X.append(text)
            Y.append(label)
    return X, Y
```

```
def word_features(tokens):
    feats = {}
    for word in tokens:
        feat = 'WORD_%s' % word
        if feat in feats:
            feats[feat] += 1
        else:
            feats[feat] = 1
    return feats
```

```
def add_features(feats, new_feats):
    for feat in new_feats:
        if feat in feats:
            feats[feat] += new_feats[feat]
        else:
            feats[feat] = new_feats[feat]
    return feats
```

This function tokenizes the document, runs all the feature extractors on it and assembles the extracted features into a dictionary mapping feature names to feature values. It is important that feature names do not conflict with each other, i.e. **different features should have different names**. Each document will have its own dictionary of features and their values.

```
def docs2features(trainX, feature_functions, tokenizer):
    examples = []
    count = 0
    for doc in trainX:
        feats = {}

        tokens = tokenizer(doc)

        for func in feature_functions:
            add_features(feats, func(tokens))

        examples.append(feats)
        count += 1

        if count % 100 == 0:
            print('Processed %d examples into features' % len(examples))

    return examples
```

This helper function converts feature names to unique numerical IDs.

```
def create_vocab(examples):
    feature_vocab = {}
    idx = 0
    for example in examples:
        for feat in example:
            if feat not in feature_vocab:
                feature_vocab[feat] = idx
                idx += 1

    return feature_vocab
```

```
# This helper function converts a set of examples from a dictionary of feature names to values representation
# to a sparse representation of feature ids to values. This is important because almost all feature values will
# be 0 for most documents and it would be wasteful to save all in memory.
```

```
def features_to_ids(examples, feature_vocab):
    new_examples = sparse.lil_matrix((len(examples), len(feature_vocab)))
    for idx, example in enumerate(examples):
        for feat in example:
            if feat in feature_vocab:
                new_examples[idx, feature_vocab[feat]] = example[feat]

    return new_examples
```

```
# Evaluation pipeline for the Logistic Regression classifier.
```

```
def train_and_test(trainX, trainY, devX, devY, feature_functions, tokenizer):
    # Pre-process training documents.
    trainX_feat = docs2features(trainX, feature_functions, tokenizer)

    # Create vocabulary from features in training examples.
    feature_vocab = create_vocab(trainX_feat)
    print('Vocabulary size: %d' % len(feature_vocab))

    trainX_ids = features_to_ids(trainX_feat, feature_vocab)

    # Train LR model.
    lr_model = LogisticRegression(penalty = 'l2', C = 1.0, solver = 'lbfgs', max_iter = 1000)
    lr_model.fit(trainX_ids, trainY)

    # Pre-process test documents.
    devX_feat = docs2features(devX, feature_functions, tokenizer)
    devX_ids = features_to_ids(devX_feat, feature_vocab)

    # Test LR model.
    print('Accuracy: %.3f' % lr_model.score(devX_ids, devY))
```

```
from google.colab import drive
drive.mount('/content/drive', force_remount=True)
```

```
Mounted at /content/drive
```

```
import os

# Use this path for Colab
datapath = '/content/drive/MyDrive/HW3/data'

train_file = os.path.join(datapath, 'imdb_sentiment_train.txt')
trainX, trainY = read_examples(train_file)

dev_file = os.path.join(datapath, 'imdb_sentiment_dev.txt')
devX, devY = read_examples(dev_file)

# Specify features to use.
features = [word_features]

# Evaluate LR model.
train_and_test(trainX, trainY, devX, devY, features, spacy_tokenizer)
```

```
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Vocabulary size: 28692
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
```

```

Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Accuracy: 0.838

```

✓ Feature engineering

Evaluate LR model performance when adding positive and negative lexicon features. We will be using Bing Liu's sentiment lexicons from <https://www.cs.uic.edu/~liub/FBS/sentiment-analysis.html>

✓ Read the positive and negative sentiment lexicons (5p)

There should be 2006 entries in the positive lexicon and 4783 entries in the positive lexicon.

```

def read_lexicon(filename):
    lexicon = set()
    with open(filename, mode = 'r', encoding = 'ISO-8859-1') as file:
        for line in file:
            # Strip whitespace from the line
            clean_line = line.strip()
            # Only add the word if the line is not empty AND doesn't start with a semicolon
            if clean_line and not clean_line.startswith(';'):
                lexicon.add(clean_line)
    return lexicon

# Add appropriate code for Colab here
lexicon_path = '/content/drive/MyDrive/HW3/data/blui'

poslex_file = os.path.join(lexicon_path, 'positive-words.txt')
neglex_file = os.path.join(lexicon_path, 'negative-words.txt')

poslex = read_lexicon(poslex_file)
neglex = read_lexicon(neglex_file)

print(len(poslex), 'entries in the positive lexicon.')
print(len(neglex), 'entries in the negative lexicon.')

2006 entries in the positive lexicon.
4783 entries in the negative lexicon.

```

✓ Use the lexicons to create two lexicon features (15p)

- A feature 'POSLEX' whose value indicates how many tokens belong to the positive lexicon.
- A feature 'NEGLEX' whose value indicates how many tokens belong to the negative lexicon.

```
def two_lexicon_features(tokens):
    feats = {'POSLEX': 0, 'NEGLEX': 0}

    # Iterate through each token in the document
    for word in tokens:
        # Convert to lowercase to ensure it matches the lowercase words in the lexicons
        word_lower = word.lower()

        # Increment the positive count if the word is in the positive lexicon
        if word_lower in poslex:
            feats['POSLEX'] += 1

        # Increment the negative count if the word is in the negative lexicon
        if word_lower in neglex:
            feats['NEGLEX'] += 1

    return feats
```

Evaluate the LR model using the two new lexicon features. Expected accuracy is around 83.8%.

```
# Specify features to use.
features = [word_features, two_lexicon_features]

# Evaluate LR model.
train_and_test(trainX, trainY, devX, devY, features, spacy_tokenizer)
```

```
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Vocabulary size: 28694
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Accuracy: 0.833
```

✓ Create a separate feature for each word that appears in each lexicon (20p)

- If a word from the positive lexicon (e.g. 'like') appears N times in the document (e.g. 5 times), add a positive lexicon feature 'POSLEX_word' for that word that is associated that value (e.g. {'POSLEX_like': 5}).
- Similarly, if a word from the negative lexicon (e.g. 'dislike') appears N times in the document (e.g. 5 times), add a negative lexicon feature 'NEGLEX_word' for that word that is associated that value (e.g. {'NEGLEX_dislike': 5}).

```
def lexicon_features(tokens):
    feats = {}

    # Iterate through each token in the document
    for word in tokens:
        # Lowercase the word so it matches the lexicon format
        word_lower = word.lower()

        # If the word is in the positive lexicon, create/update its specific feature
```

```

if word_lower in poslex:
    feat_name = 'POSLEX_' + word_lower
    # .get() safely handles the first time we see a word by defaulting to 0
    feats[feat_name] = feats.get(feat_name, 0) + 1

# If the word is in the negative lexicon, create/update its specific feature
if word_lower in neglex:
    feat_name = 'NEGLEX_' + word_lower
    feats[feat_name] = feats.get(feat_name, 0) + 1

return feats

```

Evaluate the LR model using the new per-lexicon word features. Expected accuracy is around 83.9%.

```

# Specify features to use.
features = [word_features, lexicon_features]

# Evaluate LR model.
train_and_test(trainX, trainY, devX, devY, features, spacy_tokenizer)

```

```

Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Vocabulary size: 31799
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Accuracy: 0.845

```

> Add document length feature (5p)

Add a feature 'DOC_LEN' whose value is the natural logarithm of the document length (use *math.log* to compute logarithms).

↳ 3 cells hidden

✓ Add deictic features (15p)

Add a feature 'DEICTIC_COUNT' that counts the number of 1st and 2nd person pronouns in the document.

```

def deictic_feature(tokens):
    pronouns = set(('i', 'my', 'me', 'we', 'us', 'our', 'you', 'your'))
    count = 0

    # Iterate through each token in the document
    for word in tokens:
        # Lowercase the word to ensure 'I', 'My', 'We', etc., match the set
        if word.lower() in pronouns:
            count += 1

    return {'DEICTIC_COUNT': count}

```

Evaluate the LR model using the deictic features. Expected accuracy is around 84.2%.

```
# Specify features to use.
features = [word_features, lexicon_features, len_feature, deictic_feature]

# Evaluate LR model.
train_and_test(trainX, trainY, devX, devY, features, spacy_tokenizer)
```

```
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Vocabulary size: 31801
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Accuracy: 0.845
```

Let's try without the word features. Expected accuracy is around 80.4%.

```
# Specify features to use.
features = [lexicon_features, len_feature, deictic_feature]

# Evaluate LR model.
train_and_test(trainX, trainY, devX, devY, features, spacy_tokenizer)
```

```
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
Processed 1500 examples into features
Vocabulary size: 3109
Processed 100 examples into features
Processed 200 examples into features
Processed 300 examples into features
Processed 400 examples into features
Processed 500 examples into features
Processed 600 examples into features
Processed 700 examples into features
Processed 800 examples into features
Processed 900 examples into features
Processed 1000 examples into features
Processed 1100 examples into features
Processed 1200 examples into features
Processed 1300 examples into features
Processed 1400 examples into features
```

Processed 1500 examples into features
Accuracy: 0.799

✓ Analysis ## (10p)

Include an analysis of the results that you obtained in the experiments above.

Error analysis: Do some basic error analysis where you try to explain the mistakes that the best model made and provide ideas for possible features that would alleviate these mistakes.

[BONUS] Interpretability (10p): From each class of features take 2 features that you think should be strongly correlated with the positive or negative label, and determine if the model learned a corresponding parameter that correctly expresses this correlation. For example, the feature 'WORD_loved' is expected to be very correlated with the positive label, as such the model should learn a corresponding large positive weight.

- Hint: for this, you may consider using the `coef_` attribute of the `LogisticRegression` class.

TT B I <> ↺ ↻ 🖨️ 🔍 ⌵ ⌶ — Ψ 😊 ☰ Close

Analysis of Results

Based on the results of the experiments, the steady increase to the addition of specialized features that provide the model domain-specific "shortcuts" beyond raw word counts.

```
* **Baseline:** Achieved an accuracy of 83.8% with `word_features` feature.
* **Two Lexicon:** Accuracy shifted to 83.3%.
* **Per-Word Lexicon:** Accuracy improved to 84.5%.
* **Document Length:** Accuracy of 84.4%.
* **Deictic Feature:** Accuracy of 84.5%.
* **Lexicon Only (No Words):** Accuracy of 79.9%.
```

Error Analysis

Even the best models still make mistakes. Some of the reasons for inaccuracy include:

```
* **Negation Handling:** The model currently doesn't understand "I didn't like x" can be seen as positive because of the word "like". A fix would be adding N-grams that can handle flipping the sentiment following "not" or "never".
* **Sarcasm:** Positive words are often used in a negative context. Implementing a bi-directional model could help capture this context.
* **Weight Distribution:** Currently, words like "good" and "amazing" have similar weights. A good fix would be adding weighted lexicons so that stronger words in the same class carry heavier weights.
```

Analysis of Results

Based on the results of the experiments, the steady increase in accuracy is due to the addition of specialized features that provide the model with domain-specific "shortcuts" beyond raw word counts.

- **Baseline:** Achieved an accuracy of 83.8% with `word_features` as the only feature.
- **Two Lexicon:** Accuracy shifted to 83.3%.
- **Per-Word Lexicon:** Accuracy improved to 84.5%.
- **Document Length:** Accuracy of 84.4%.
- **Deictic Feature:** Accuracy of 84.5%.
- **Lexicon Only (No Words):** Accuracy of 79.9%.

Error Analysis

Even the best models still make mistakes. Some of the reasons for the inaccuracy include:

- **Negation Handling:** The model currently doesn't understand negation (e.g., "I didn't like x" can be seen as positive because of the word "like"). A fix would be adding N-grams that can handle flipping the sentiment of words following "not" or "never".
- **Sarcasm:** Positive words are often used in a negative context, or vice versa. Implementing a bi-directional model could help capture this context.
- **Weight Distribution:** Currently, words like "good" and "amazing" might have similar weights. A good fix would be adding weighted lexicons so that stronger words in the same class carry heavier weights.

```
import sys
import os

# Explicitly use all feature extractors for a comprehensive analysis
all_features = [word_features, two_lexicon_features, lexicon_features, len_feature, deictic_feature]

# Temporarily suppress stdout to avoid truncated output from the progress prints
old_stdout = sys.stdout
sys.stdout = open(os.devnull, 'w')

# feature_vocab and lr_model in scope for analysis
trainX_feat = docs2features(trainX, all_features, spacy_tokenizer)
feature_vocab = create_vocab(trainX_feat)
trainX_ids = features_to_ids(trainX_feat, feature_vocab)
```

```
lr_model = LogisticRegression(penalty='l2', C=1.0, solver='lbfgs', max_iter=1000)
lr_model.fit(trainX_ids, trainY)

# Restore stdout
sys.stdout = old_stdout

# Helper function to check feature weight
def check_weight(feature_name):
    if feature_name in feature_vocab:
        idx = feature_vocab[feature_name]
        weight = lr_model.coef_[0][idx]
        print(f"Weight for '{feature_name}': {weight:.4f}")
    else:
        print(f"Feature '{feature_name}' not found in vocabulary.")

print("--- WORD FEATURES ---")
check_weight('WORD_excellent')
check_weight('WORD_terrible')

print("\n--- LEXICON FEATURES ---")
check_weight('POSLEX')
check_weight('NEGLEX')

print("\n--- STRUCTURAL FEATURES ---")
check_weight('DOC_LEN')
check_weight('DEICTIC_COUNT')
```

```
--- WORD FEATURES ---
Weight for 'WORD_excellent': 0.4217
Weight for 'WORD_terrible': -0.2962

--- LEXICON FEATURES ---
Weight for 'POSLEX': 0.4250
Weight for 'NEGLEX': -0.3617

--- STRUCTURAL FEATURES ---
Weight for 'DOC_LEN': -0.0574
Weight for 'DEICTIC_COUNT': 0.1409
```